



EL2805 Reinforcement Learning

Exercise Session 3

November 17th 2025

Division of Decision and Control Systems
School of Electrical Engineering and Computer Science
KTH Royal Institute of Technology

Instructions.

- These exercises are for your practice, and you do not need to submit solutions. We encourage you to work on them before the corresponding exercise session on Nov 24th. The materials for the 4th exercise session will be available on Nov 24th.
- In each exercise session, we will go through a selection of topics chosen by the teaching assistant and participating students.
- Homework and labs will be based on concepts covered in lectures and exercise sessions. These materials are excellent preparation for your graded assignments!
- There is no need to memorize equations - relevant materials will be provided during the exam. Focus on concepts and understanding!
- The materials cover a wide range of topics, so feel free to focus on what interests you most. However, to develop a well-rounded understanding of RL, it is important that you engage with both theoretical and practical exercises.
- We use green boxes to highlight established results and definitions, yellow for problems that require your solutions, and pink for high-level remarks. Exercises marked with a ★ are more challenging and intended for students seeking an extra challenge.
- We value your feedback on the materials provided. If you have any questions, concerns, or suggestions, do not hesitate to reach out to any of the TAs.

From Tabular to Deep: Extending Q -learning

*“We present the **first deep learning model** to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning.”*

– DQN paper [1] (2013)

In this exercise session, we will study DQN (Deep Q-Network), an algorithm based on Q -learning that enabled the first deep RL model. In addition to the lecture notes, we encourage you to consult books on deep RL, such as [2], available on learning.oreilly.com, for more details on DQN and other deep RL algorithms.

We begin by revisiting some properties of tabular SARSA and Q -learning. Then, we will explore three fundamental challenges in deep RL and the solutions proposed in DQN. Finally, we will implement key parts of DQN algorithm.

1.1 A Warm-Up: Do Policies Learned by SARSA and Q -learning Differ?

Recall that the updates for both Q -learning and SARSA at time step $t + 1$ are given by:

$$Q^{(t+1)}(s, a) = Q^{(t)}(s, a) + \mathbf{1}_{(s_t, a_t)=(s, a)} \alpha_t (\text{target}_t - Q^{(t)}(s_t, a_t))$$

with $\text{target}_t = r_t + \gamma \max_{b \in \mathcal{A}} Q^{(t)}(s_{t+1}, b)$ (Q -learning) and $\text{target}_t = r_t + \gamma Q^{(t)}(s_{t+1}, a_{t+1})$ (SARSA).

Next, we consider the FrozenLake environment with a 3×3 lake and a single hole (shown in the right figure). Our goal is to analyze the policies learned by Q -learning and SARSA.

Exercise 1. You run the Q -learning and SARSA algorithms with zero-initialization, parameters $\varepsilon = 0.1$, $\alpha = 0.5$, and $\gamma = 1$ for 2 episodes, observing the following samples: $(s_t, a_t, r_t, \dots)$: $(6, \uparrow, -1, 3, \rightarrow, -1, 4, \downarrow, -1000, 7)$, $(6, \uparrow, -1, 3, \rightarrow, -1, 4, \downarrow, -1000, 7)$. Write down the Q -values for both algorithms after these two episodes.



Recall that Q -learning is an off-policy control method, while SARSA is on-policy. This distinction means:

- SARSA learns the action-value function Q^π for the currently active policy π . If the collected samples follow policy π , SARSA updates its estimates for that specific policy.
- Q -learning, on the other hand, directly aims to estimate Q^* , the optimal action-value function. Its targets are based on $\max_{b \in \mathcal{A}} Q^{(t)}(s_{t+1}, b)$, meaning it considers the maximizing action regardless of the current policy.

This fundamental difference leads the two methods to propose different policies under certain conditions. Let's consider what happens when the agent is in state 3 on the map above. With $\varepsilon = 0.1$, there is a 10% chance of ending up in the hole when in state 4, even if the agent knows that the "down" action is risky. To avoid this danger, the agent might choose to move "up" in state 3, staying further from the hole. One of the two methods adopts this safer strategy - which one do you think it is?

Exercise 2. You got tired of calculating Q -values by hand and have implemented Q -learning and SARSA. However, you forgot to write the update rules in the script `safeSARSA.py`. Complete the missing code, and run the script for 10,000 episodes. Examine the learned Q -values and try to explain the difference in Q -estimates for state 3. Explain why SARSA picks the safer, longer path, while Q -learning chooses the optimal, but riskier path.

1.2 [solve] Challenge 1: Maximization Bias and Double Q-learning

In machine learning, we often rely on the idea that averaging a large amount of noisy data will yield accurate estimates of the true quantities of interest. However, in some cases, estimators can exhibit bias, meaning they do not converge to the desired values. One such issue can arise in Q -learning, and we illustrate it in the exercise below.

Exercise 3. You choose daily between two restaurants, X and Y . Each day you observe independent ratings $x_i, y_i \in \{1, 2, 3, 4, 5\}$, which are unbiased samples of their true mean qualities μ_X and μ_Y . You always visit the restaurant with the higher observed rating $z_i = \max\{x_i, y_i\}$. After N visits, your estimated quality of the better restaurant becomes $\frac{1}{N} \sum_{i=1}^N z_i \rightarrow \mathbb{E}z_i$. Given that the ratings follow these distributions:

$$\mathbb{P}(x_i = k) = \frac{1}{5} \text{ for } k = 1, 2, 3, 4, 5,$$

$$\mathbb{P}(y_i = k) = \frac{1}{9} \text{ for } k = 1, 2, 3, \text{ and } \mathbb{P}(y_i = k) = \frac{1}{3} \text{ for } k = 4, 5,$$

show that this procedure likely overestimates the quality of best restaurant by showing:

$$\max\{\mu_X, \mu_Y\} = \max\{\mathbb{E}x_i, \mathbb{E}y_i\} \leq \mathbb{E}\max\{x_i, y_i\} = \mathbb{E}z_i$$

△Hints

Derive distribution for $z = \max\{x, y\}$ given distributions of x and y by considering cases.

This overestimation phenomenon is called maximization bias. It arises when estimating $\mathbb{E}\max\{X, Y\}$, instead of $\max\{\mathbb{E}X, \mathbb{E}Y\}$. A similar phenomenon occurs in Q -learning when targeting the update $r_t + \gamma \max_{b \in \mathcal{A}} Q^{(t)}(s_{t+1}, b)$. Namely, the target can be rewritten as follows:

$$r_t + \gamma \max_{b \in \mathcal{A}} Q^{(t)}(s_{t+1}, b) = r_t + \gamma Q^{(t)}(s_{t+1}, \arg\max_{b \in \mathcal{A}} Q^{(t)}(s_{t+1}, b))$$

and thus the bias occurs because the same $Q^{(t)}$ values are used both to find the best action $a^* = \arg\max_{b \in \mathcal{A}} Q^{(t)}(s_{t+1}, b)$ and to evaluate its value $Q^{(t)}(s_{t+1}, a^*)$. To mitigate this bias, it is usually suggested to use two independent estimators of the $Q^{(t)}$ function: $Q_1^{(t)}$ and $Q_2^{(t)}$.

Exercise 4. Let s_{t+1} be a **fixed** state. Suppose both estimators are unbiased for every action a : $\mathbb{E}[Q_1^{(t)}(s_{t+1}, a)] = \mathbb{E}[Q_2^{(t)}(s_{t+1}, a)] = \mathbb{E}[Q^{(t)}(s_{t+1}, a)]^1$. Let $a^* \in \arg\max_b Q_1^{(t)}(s_{t+1}, b)$ be the action that looks optimal under the first estimator. Show that its value under the second estimator satisfies:

$$\mathbb{E}[Q_2^{(t)}(s_{t+1}, a^*)] \leq \max_a \mathbb{E}[Q^{(t)}(s_{t+1}, a)].$$

△Hints

1. Apply law of total expectation to $\mathbb{E}[Q_2^{(t)}(s_{t+1}, a^*)]$, conditioning on optimality of a^* .
2. If a^* is optimal, use the fact that the estimators are unbiased.
3. If a^* is not optimal, further use that the optimal action has strictly higher Q -value.

While this approach does not produce an unbiased approximation of the maximum expected value, it avoids the overestimation seen in standard Q -learning. Instead, it introduces a slight underestimation, which is often desirable in reinforcement learning. Underestimation typically leads to more stable and robust behavior, whereas overestimation can result in the agent taking poor actions because it incorrectly believes them to be optimal.

★ **Further reading.** This insight motivated the development of **Double Q-learning** [3]. The algorithm maintains two separate Q -estimates. It selects the maximizing actions based on one estimate and evaluates them using the other. By decoupling action selection from value evaluation, Double Q -learning reduces maximization bias and improves stability. Further adaptation of this idea to deep RL has been explored in [4].

¹Expectations here are over the randomness in the estimate $Q^{(t)}$ i.e., over $Q^{(0)}, s_1, a_1, r_1, s_2, \dots, s_t, a_t, r_t$, while s_{t+1} is fixed.

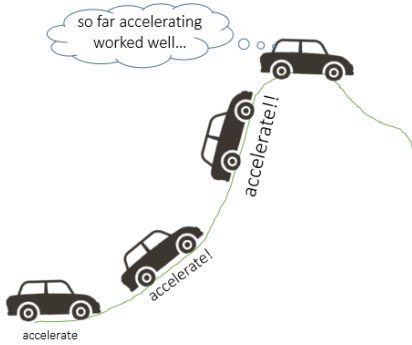
1.3 [code] Challenge 2: Correlations and Replay Buffers

Since DQN is based on Q -learning, it follows a similar sampling strategy based on ϵ -greedy exploration policy. This strategy selects greedy actions based on the current estimates $Q^{(t)}$ with probability $1 - \epsilon$, and otherwise takes a random action.

However, the trajectories used for Q -learning are Markov chains, where the next state s_{t+1} is sampled from the transition distribution $P(\cdot|s_t, a_t)$. As a result, successive sample tuples (s_t, a_t, r_t, s_{t+1}) are correlated.

This violates a key assumption in many statistical studies of RL algorithms: the independence of samples. Correlated data can make training unstable, leading to poor convergence or failure to learn effectively.

To address this issue, DQN introduces a simple yet powerful mechanism: the Experience Replay Buffer. The replay buffer is a FIFO (first-in-first-out) queue that stores observed sample tuples (s_t, a_t, r_t, s_{t+1}) . Instead of immediately updating the Q -estimates with each new sample, the algorithm periodically samples a mini-batch (typically 32 or 64 tuples) uniformly at random from the buffer. Updates to the Q -network are then performed using this shuffled batch of experiences. By mixing older and newer experiences, the replay buffer reduces correlations between samples, stabilizing training and improving the network's generalization. We illustrate this next.



Exercise 5. A company developing autonomous driving systems has encountered failures in its DQN training pipeline after removing the replay buffer: vehicles accelerate uncontrollably on downhill segments.

Inspect the script `downhill.py`, implement a replay buffer, and evaluate whether it resolves the issue. Explain how the replay buffer affects training stability.

Why are Replay Buffers Essential? One of the main challenges in deep reinforcement learning is the tendency of neural networks to experience **catastrophic forgetting** - abruptly and drastically overwriting previously learned information when exposed to new data. The replay buffer mitigates this issue by ensuring that updates are based on a diverse mix of both older and newer experiences, helping the agent retain past knowledge while learning from new observations.

1.4 [code] Challenge 3: Moving Targets and Target Networks

In DQN, instead of updating every state-action pair as in traditional Q -learning, we parametrize the Q -function using a neural network with parameters θ . The update rule minimizes the squared Bellman error over a mini-batch:

$$J(\theta) = \frac{1}{B} \sum_{i=1}^B (r_i + \gamma \max_b Q_{\theta}(s_{i+1}, b) - Q_{\theta}(s_i, a_i))^2$$

where B is the size of the mini-batch. The parameters θ are updated via gradient descent $\theta \leftarrow \theta - \alpha \nabla_{\theta} J(\theta)$. However, this update can be unstable. The problem arises because the target $r_i + \gamma \max_b Q_{\theta}(s_{i+1}, b)$ depends on the current parameters θ . Updating θ while simultaneously using θ to compute the target creates a *moving target*, which can lead to oscillations or divergence.

To see this concretely, consider a linearized example. Suppose we have a 2-dimensional parameter vector $\theta = (\theta_1, \theta_2)^{\top}$ and a linear update rule:

$$\begin{aligned} \theta_1 &\leftarrow b_1 + a_{11}\theta_1 + a_{12}\theta_2, \\ \theta_2 &\leftarrow b_2 + a_{21}\theta_1 + a_{22}\theta_2, \end{aligned}$$

where b_i and a_{ij} are constants. This models the effect of bootstrapping on the current parameters: each component of θ is updated using a linear combination of the previous parameters.

Exercise 6. Given this 2-d system, determine the conditions on $A = [a_{ij}]$ under which the system converges to a fixed point, oscillates, or diverges. In `moving_targets.py`, compute the fixed point $\theta^* = (I - A)^{-1}b$ and update parameters for three different cases of A . Discuss when and why oscillations or divergence occur, and when the updates lead to the fixed point solution.

△Hints

From linear system theory, systems are stable if all eigenvalues of A have norm < 1 .

Stabilizing updates with target networks. To address this instability, DQN separates the parameters used for computing the target from those being updated. Specifically, gradients are only taken with respect to θ , while the target uses a fixed copy of the parameters, denoted $\bar{\theta}$. The update rule becomes:

$$\theta \leftarrow \theta + \alpha \underbrace{(r_t + \gamma \max_b Q_{\bar{\theta}}(s_{t+1}, b) - Q_{\theta}(s_t, a_t))}_{\text{target}_t} \nabla_{\theta} Q_{\theta}(s_t, a_t)$$

Instead of updating the target after each iteration, it can be updated periodically, every K iterations. Alternatively, we can use soft-averaging (Polyak averaging) which updates $\bar{\theta}$ as a weighted average of the current target parameters and the updated parameters: $\bar{\theta} \leftarrow (1 - \tau)\bar{\theta} + \tau\theta$, where parameter $\tau \in (0, 1)$ controls the averaging rate.

Exercise 7. In `moving_targets.py` implement K -step target updates and Polyak averaging. Comment on the stability of the updates and the values to which parameters converge. What happens to the eigenvalues of the system when a fixed target is used?

1.5 [code] Coding first deep RL model using DQN

Understanding DQN is crucial, as many of the state-of-the-art deep RL algorithms build upon its principles. In this exercise, you will implement key components of DQN to learn a balancing policy for the CartPole environment. Follow these steps to train your first DQN-based agent:

1. Open and run the file `DQNelements.py`. Get acquainted with the code. You should observe a CartPole being controlled by a random agent.
2. Implement an Experience Replay Buffer, where each tuple (s, a, r, s', d) contains: current state s , taken action a , received reward r , next state s' , and a boolean variable d indicating if the episode terminated. Complete the missing parts in the class `ExperienceReplayBuffer`.
3. Implement the class `MyNetwork` to define your neural network architecture. Follow the guidelines provided in the script for this step.
4. Use the neural network you just defined to select actions for the CartPole environment.
5. Set up an optimizer and perform backpropagation to update the network's parameters.
6. Use TensorBoard (<https://www.tensorflow.org/tensorboard>) to monitor training.

References

- [1] Volodymyr Mnih. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [2] Sudharsan Ravichandiran. *Deep Reinforcement Learning with Python: Master classic RL, deep RL, distributional RL, inverse RL, and more with OpenAI Gym and TensorFlow*. Packt Publishing Ltd, 2020.
- [3] Hado Hasselt. Double q-learning. *Advances in neural information processing systems*, 23, 2010.
- [4] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.